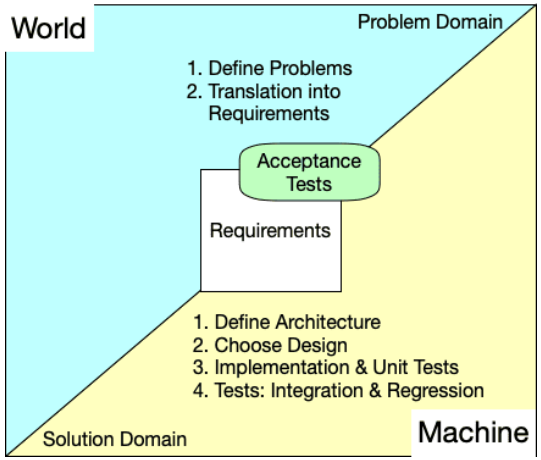
Project

Solving Problems **Individually or as a Team**

From Problem to Software Solution

1. **World → Requirements** — understand the problem, write testable requirements
2. **Requirements → Design** — decide the architecture (high level) and the module/interface design (low level)
3. **Design → Implementation** — build it, with unit tests and docs along the way
4. **Implementation → Validation** — prove the requirements are actually met

The Big Picture

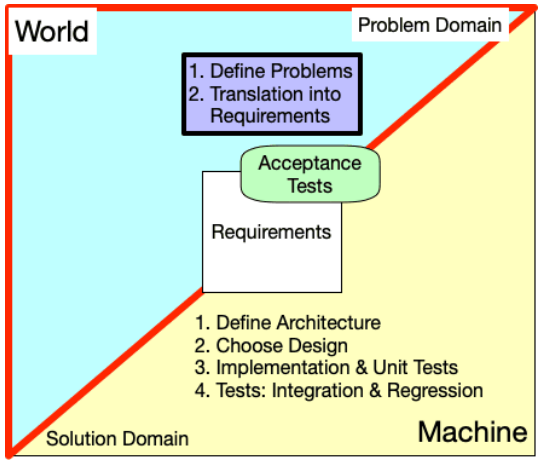


We use a small `TodoApp` to make the process concrete. The same steps apply to larger projects.

Step 1: World → Requirements

World = the real problem and its environment, before any software exists.

First, understand the problem clearly — and why it matters.



Understand Why (Why do we need this software?)

Because students juggle many deadlines across courses. They often:

- Forget upcoming deadlines
- Start assignments too late
- Lose track of multiple schedules

They need a tool that helps them manage their time and avoid missing deadlines.

Define Problems: What exactly needs solving?

P1: No Proactive Notifications

Students have no system that warns them before a deadline.

P2: Limited Accessibility

Students do most of their coursework in a browser, but have no browser-based tool to manage schedules across devices.

Translate the Problems into Features/Requirements

Once we understand the **World** (problem domain), we turn it into a feature list.

Features

Features restate the problems as things the app does:

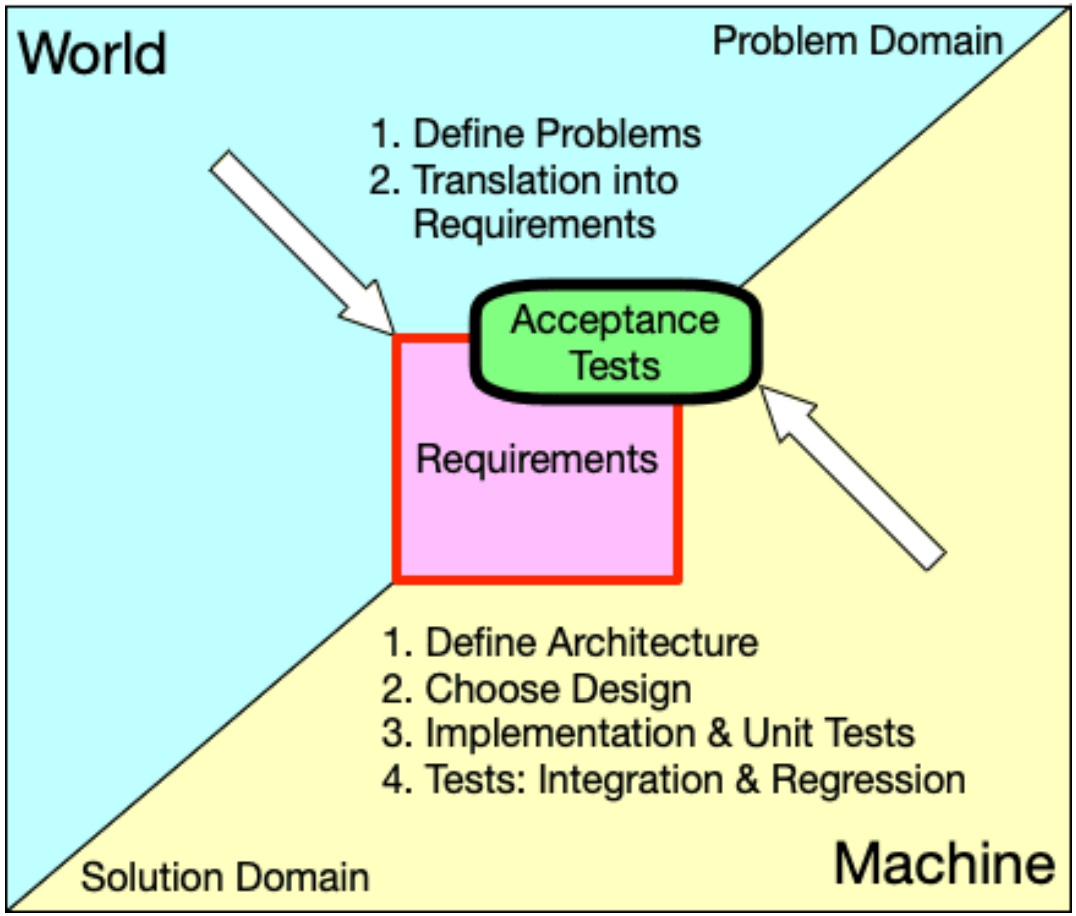
1. Notify students of a deadline N days in advance, by email.
2. Let students create, read, update, and delete (CRUD) schedules on the web.

Requirements (User Story Mapping)

Each feature breaks down into **user stories**, each with:

1. **Who** the user is
2. **What** they need to do
3. **Why** they need it

Each user story gets **acceptance criteria** — concrete conditions checked by acceptance tests.


World
Problem Domain
1. Define Problems
2. Translation into Requirements
Acceptance Tests
Requirements
1. Define Architecture
2. Choose Design
3. Implementation & Unit Tests
4. Tests: Integration & Regression
Solution Domain
Machine

Feature 1: Notify students N days before a deadline

Here, **S** stands for one schedule/task (e.g., "HW3 due Friday").

- **F1-R1**: **As** a student, I **want** an email N days before schedule S is due, **so that** I don't miss it.
- **F1-R2**: **As** a student, I **want to** mark schedule S as complete, **so that** I stop getting reminders and can track my progress.

F1-R1 is too large to implement as one story — it's an **epic**. We split it:

- **F1-R1a**: **As** a student, I **want to** set my notification email and N (days before), **so that** the system knows where and when to notify me.
- **F1-R1b**: **As** a student, I **want** the system to automatically send the email N days before the due date, **so that** I'm alerted proactively.

Feature 1 → **3 testable requirements**: F1-R1a, F1-R1b, F1-R2.

- Example criterion for F1-R1b: If an incomplete task is due in N days, send one reminder email.

Feature 2: CRUD schedules

- **F2-R1**: **As** a student, I **want to** create a schedule, **so that** it's saved online.
- **F2-R2**: **As** a student, I **want to** view saved schedules, **so that** I can track upcoming tasks.

- **F2-R3**: **As** a student, I **want to** update a schedule, **so that** my plan stays accurate.
- **F2-R4**: **As** a student, I **want to** delete a schedule, **so that** I can remove tasks I no longer need.

Two features, seven requirements — each with its own ID:

- Feature 1 → F1-R1a, F1-R1b, F1-R2 *(3)*
- Feature 2 → F2-R1, F2-R2, F2-R3, F2-R4 *(4)*

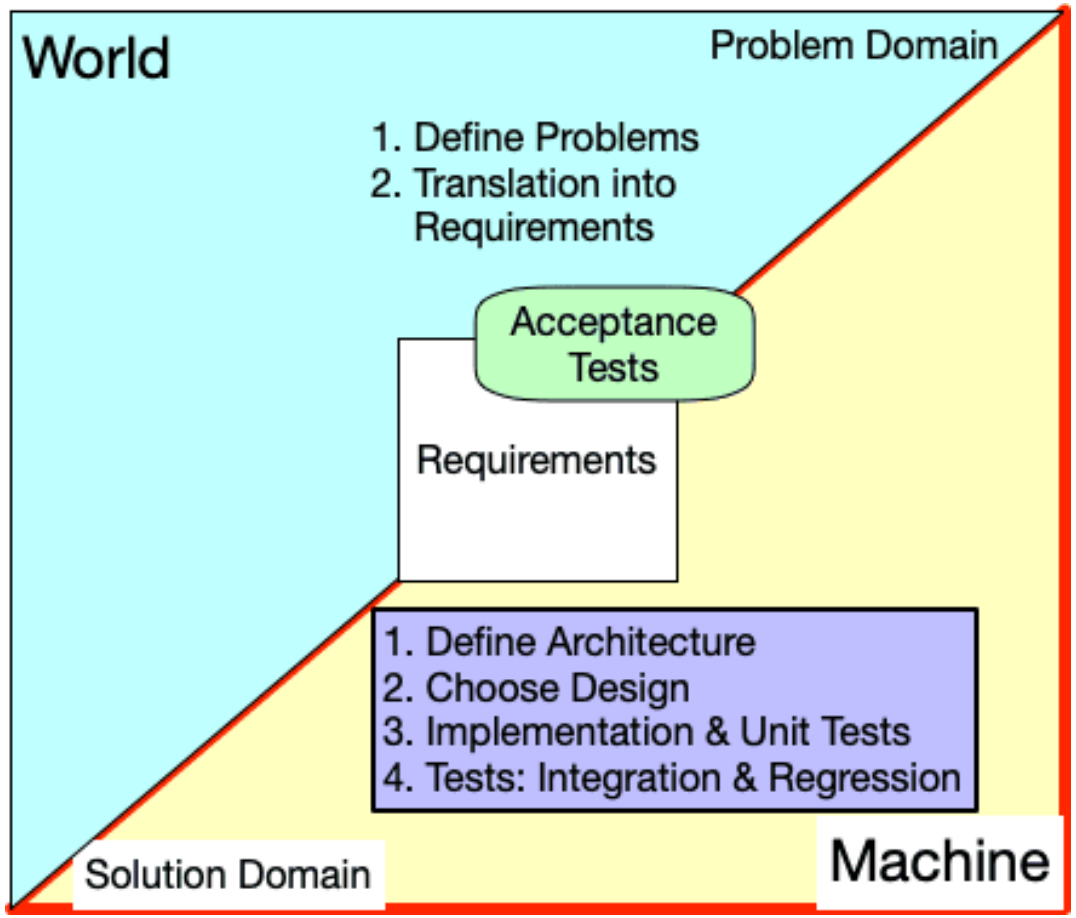
Step 2: Requirements → Design

Choosing the Machine (Solution Domain)

Machine = the software system we build to solve the problem.

We know the problem, and we've translated it into requirements as user stories. Next:

- Architect and design the Machine (software)
- Implement the design
- Verify the architecture and design are properly implemented


World
Problem Domain
1. Define Problems
2. Translation into Requirements
Acceptance Tests
Requirements
1. Define Architecture
2. Choose Design
3. Implementation & Unit Tests
4. Tests: Integration & Regression
Solution Domain
Machine

From Requirements to Architecture

Before picking an architecture, map each requirement to what it needs:

- Browser access (F2) → a **web client**
- CRUD schedules (F2) → a **server API** + **database**
- Email reminders (F1) → a **scheduler** + **email service**

A good architecture gives every requirement a home.

Select Architecture

We choose a **3-Tier** web architecture, plus a background job for reminders:

- The client reaches the server over HTTP.
- The server handles requests through a REST API.
- The server uses a database for CRUD operations.
- A **scheduler** checks due dates and triggers the **email service**.

```
Web Client → REST API → Database
Scheduler → Email Service
```

Data Modeling

- Architecture shows how information flows through the system.
- We also need to define the information itself — this is `data modeling`.
- The model must cover every field our requirements actually need.

In `TodoApp`, each schedule needs:

1. **Title** — the action *(F2)*
2. **Date** — when it's due *(F2)*
3. **Email** — where to send reminders *(F1-R1a)*
4. **Reminder Days (N)** — days before the deadline to notify *(F1-R1a)*
5. **Completed** — done or not *(F1-R2)*

(To keep this toy app simple, we skip a login system — each schedule stores its own notification email and N.)

Every part of our design exists to create, process, store, and delete this data.

Choose Design

In this course, **design** simply means **modules & interfaces**:

- Build modules with clear interfaces
- Unit-test each module's important behavior
- Show exactly how our data model gets used

(Real-world design also draws on OOP principles like SOLID and tools like UML — you'll go deeper into those in ASE 420.)

Front End Design

Design is mainly modules and interfaces, but the front end also needs a usable UI.

- A `views` directory holds all the HTML (GUI) components.

```
── views
    ├── detail.ejs
    ├── edit.ejs
    ├── list.ejs
    ├── nav.ejs
    └── write.ejs
```

Utility Functions & Tests

Building applications requires utility functions — creating them is a key part of software design.

- Build utility modules with clear interfaces other modules can reuse.
- Test each module's important behavior through its interface.

```
├── tests
│   └── db.util.js
├── util
│   └── util.js
...
```

Main Modules

Requests flow in one direction:

`View → Route → API/Service → Database`

```
├── index.js
├── api
│   ├── routes.js
│   └── api.js
```

Step 3: Design → Implementation

With architecture and design in hand, we can implement — and write tests as we go.

Choosing the Technology Stack

Once we know the **why** (problem) and the **what** (user stories), choosing the **how** gets much simpler:

- Pick a stack suited to the problem.
- Prefer popular tools that are easy to learn and use.

For our problem and requirements, we use a common web stack:

- **JavaScript** as the language
- **Node.js** as the runtime
- **MongoDB** as the database
- A scheduler and email service for reminders

Unit Tests

For each module, we check its important behavior.

- This is called a **unit test**.
- Most language ecosystems provide testing libraries or frameworks.

Step 4: Implementation → Validation

Now we check each requirement's acceptance criteria, supported by integration and unit tests.

Kinds of Tests

Tests differ along two dimensions.

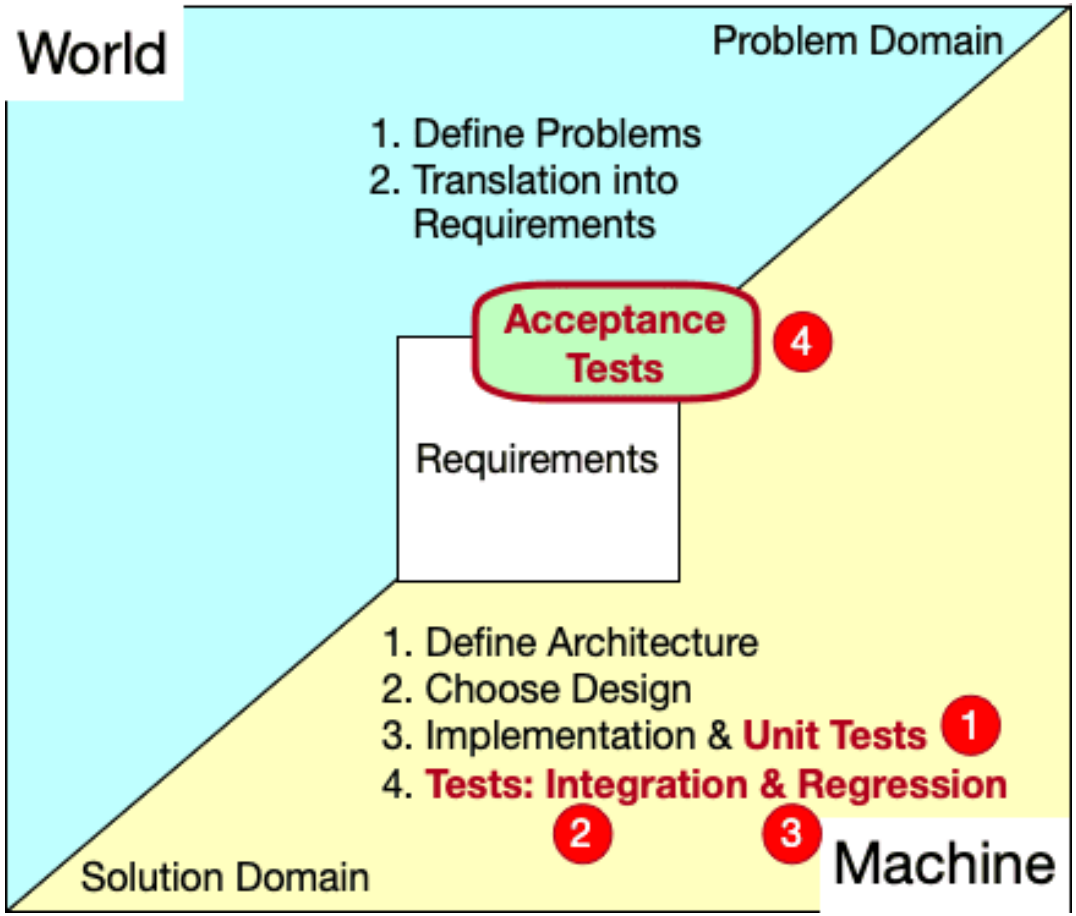
By scope (how much of the system is tested)

- Unit: one module
- Integration: several modules together
- Acceptance: the whole system, against a requirement

By purpose

- Regression: rerun existing tests to check new changes didn't break anything — any test above can be a regression test.

- A solid project combines unit, integration, and acceptance tests.
- and reruns them after changes to catch regressions.



Documentation

Documentation preserves and shares project knowledge.

Good docs answer:

1. What's the goal of this project?
2. How is the project going?
3. What's the architecture & design?
4. Where are the source, tests, and other documents?
5. How do I use this software?

Three Kinds of Documentation

User docs — for end users: how to use the system (steps, screenshots, troubleshooting)

Design docs — for developers: how the system works (architecture, APIs, data model, key decisions)

Process docs — for the team: how the work is going (backlog, sprint notes, progress reports)

Agile Keeps Only What Helps

Agile avoids heavy paperwork, but it doesn't skip documentation — it keeps only what earns its place.

- Long documents go stale fast
- Requirements change frequently
- Talking and prototyping are often faster
- Documentation should support the work, not slow it down

For This Course, You Must Document

Design docs

- Features & requirements (user stories + acceptance criteria)
- Architecture diagram & data model
- Directory structure

Process docs

- A backlog (done / in progress / planned)
- Short progress notes for your team & instructor

Tests as Design Documents

Well-written tests double as a **living design document**:

- They show expected behavior — input, action, outcome
- They reveal rules and edge cases
- Update them whenever behavior changes, and they'll stay accurate

Course Logistics

Related ASE Courses

- ASE 330 (HCI) — UX/UI, so users can use software effectively
- ASE 420 (Software Design) — design rules, tools, and practices
- ASE 456 (Cross-platform Development) — architecture for cross-platform apps

Project Repositories

- GitHub — architecture & design docs, source code, tests
- Canvas Pages — process docs, team progress reports

Portfolio Management

After each project, students should add the project artifacts to their portfolio.

- GitHub Pages is a common, free choice for hosting a portfolio site.
- Many engineers eventually use their own domain instead.